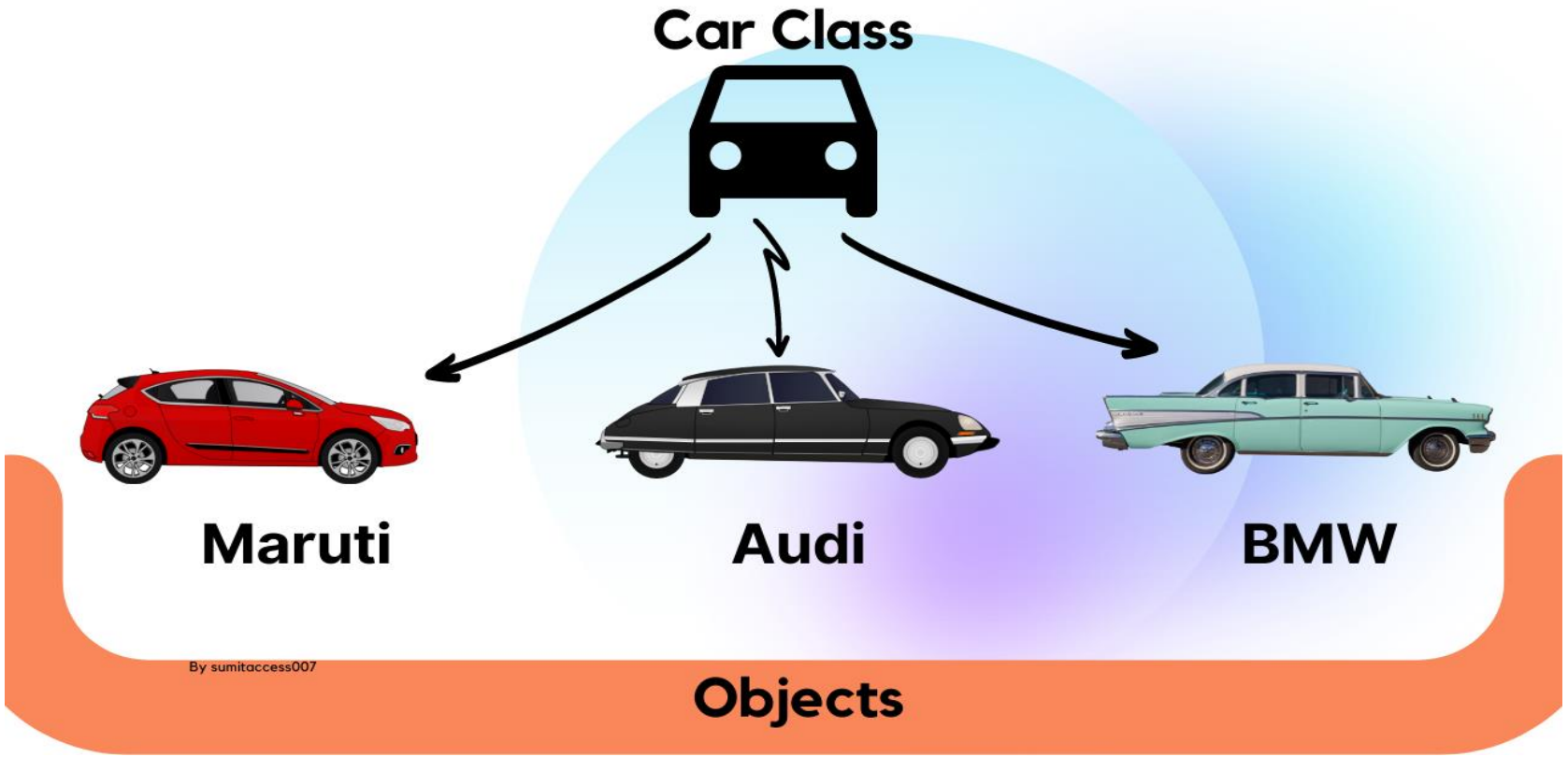
Unit :-04
"Object-Oriented Programming "

Topics List :-

- Classes, objects , Constructor and Constructor overloading.
- Access Specifiers and Encapsulation.
- Inheritance and Types of Inheritance.
- Polymorphism and Types of Polymorphism.
- Abstraction , Abstract Classes and Interfaces.
- This ,Super , Final Keywords and Object Class.

Topic –01
"Classes, Objects , Constructor , Constructor Overloading"

- **Classes :-** In OOP Class Is the Blueprint to create an object . It is used to create objects that share common properties and behavior. A class is defined using the `class` keyword, followed by the class name, access modifiers, field declarations (attributes), and methods (behaviors).



Syntax For Initialization:

```
access_modifier class ClassName
{
   data_type fieldName;
   access_modifier return_type methodName()
   {
      // code
   }
}
```

- **Objects:-** In OOP Object is Instance of Class. An object is used to access the variables and methods of a class. An object must be declared, allocated in memory using the new keyword, and then initialized via a constructor call.



Syntax of Initialization:-

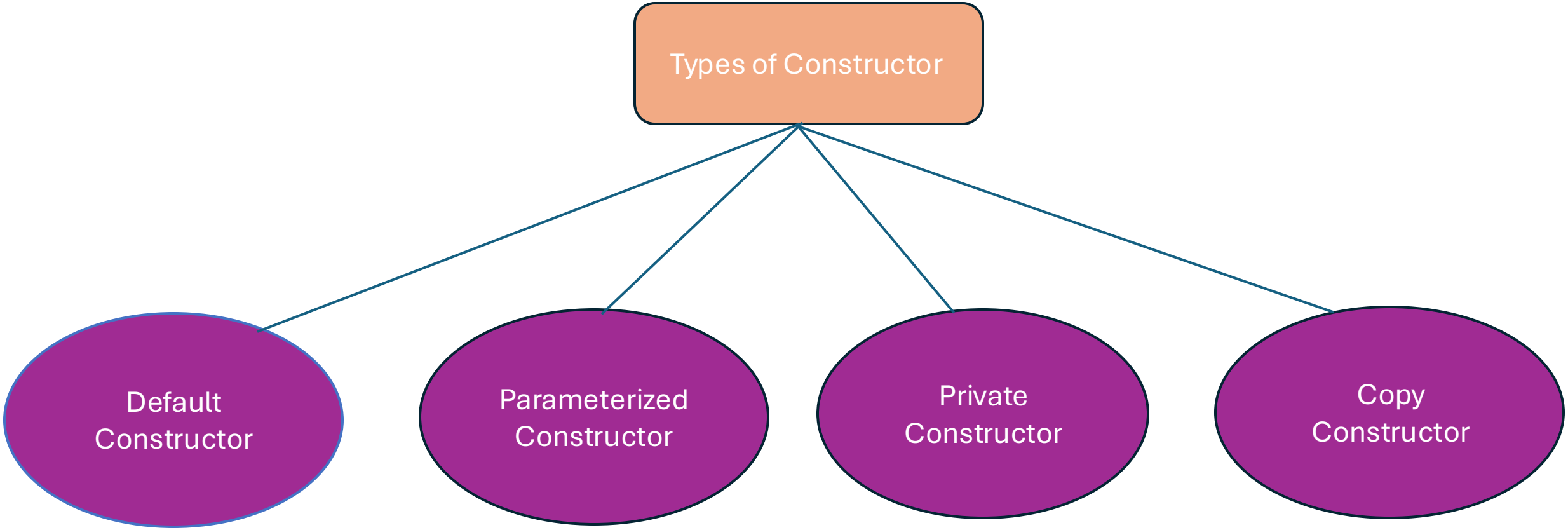
ClassName objectReferenceVar = new ClassName();

- **Constructor :-** A **constructor** in Java is a special block of code used to initialize an object when it is created. It looks like a method, but it has no return type and its name strictly matches the class name. Constructors are invoked automatically using the new keyword.

Syntax of Initialization:

```
class ClassName
{
    [AccessModifier] ClassName(parameters_list)
    {
// Initialization code (assigning values to variables)
    }
}
```

Types of Constructors :


Types of Constructor
Default Constructor
Parameterized Constructor
Private Constructor
Copy Constructor

- **Default Constructor :-** A default constructor has no parameters. It's used to assign default values to an object. If no constructor is explicitly defined, Java provides a default constructor.

Syntax of Initialization :-

```
import java.io.*;
class Constructor
{
  // Default Constructor
 Public  Constructor()
{
    System.out.println("Default constructor");
}
     public static void main(String[] args)
{
   Constructor hello = new Constructor();
}
}
```

- **Parameterized Constructor :-** A constructor that has parameters is known as parameterized constructor. If we want to initialize fields of the class with our own values, then use a parameterized constructor.

  Syntax of Initialization :-

```
class parametrizedcons
{
   // data members of the class
      String name;
     int id;
     // Parameterized Constructor
    Parametercons(String name, int id)
   {
       this.name = name;
       this.id = id;
   }
```

```
 // Method to display object data
   void display()
{
      System.out.println("Name: " + name
                  + " and Id: " + id);
   }
   public static void main(String[] args){

      // This will invoke the parameterized constructor
      parametercons p1 = new parametercons("Sweta", 68);
      p1.display();
   }
}
```

- **Private Constructor :-** A private constructor is a constructor that has the `private` access modifier applied to it. applying `private` to a constructor means no code outside of that specific class can use the `new` keyword to create an instance of it.

Syntax of Initialization :

```
class PrivConst {
// Private constructor
Private  PrivConst ()
{
   System.out.println("Private constructor called");
}
public static void displayMessage()
{
      System.out.println("Hello from display message ");
}
}
   class Main
   {
      public static void main(String[] args)
   {
        Main.displayMessage();
   }
}
```

- **Copy Constructor :-** A copy constructor is a specialized parameterized constructor designed to initiate a new object by cloning the data fields of an already existing object of the identical class type.

  Syntax of initialization :

```
class ClassName
{
  int variable;
   // Normal constructor
  ClassName(int variable)
  {
     this.variable = variable;
  }
    // Copy constructor
   ClassName(ClassName object)
   {
       this.variable = object.variable;
   }
}
```

- **Constructor overloading :-** When a class contains multiple constructors with different parameter lists, it is called Constructor Overloading.This is an example of Compile-Time Polymorphism.

Constructors must differ in:

- Number of parameters,
- Type of parameters,
- Order of parameters

Example :-

```
class Student {

   String name;
   int age;

   // Constructor 1: No parameter
   Student() {
      name = "Unknown";
      age = 0;
   }
```

```
// Constructor 2: One parameter
Student(String name) {
    this.name = name;
    age = 0;
}
// Constructor 3: Two parameters
Student(String name, int age) {
    this.name = name;
    this.age = age;
}

void display() {
    System.out.println("Name: " + name);
    System.out.println("Age: " + age);
    System.out.println("----------------");
}
```

```
 public static void main(String[] args)
{
      Student s1 = new Student();

      Student s2 = new Student("Dhruvil");

      Student s3 = new Student("Dhruvil", 22);

      s1.display();
      s2.display();
      s3.display();
   }
}
```

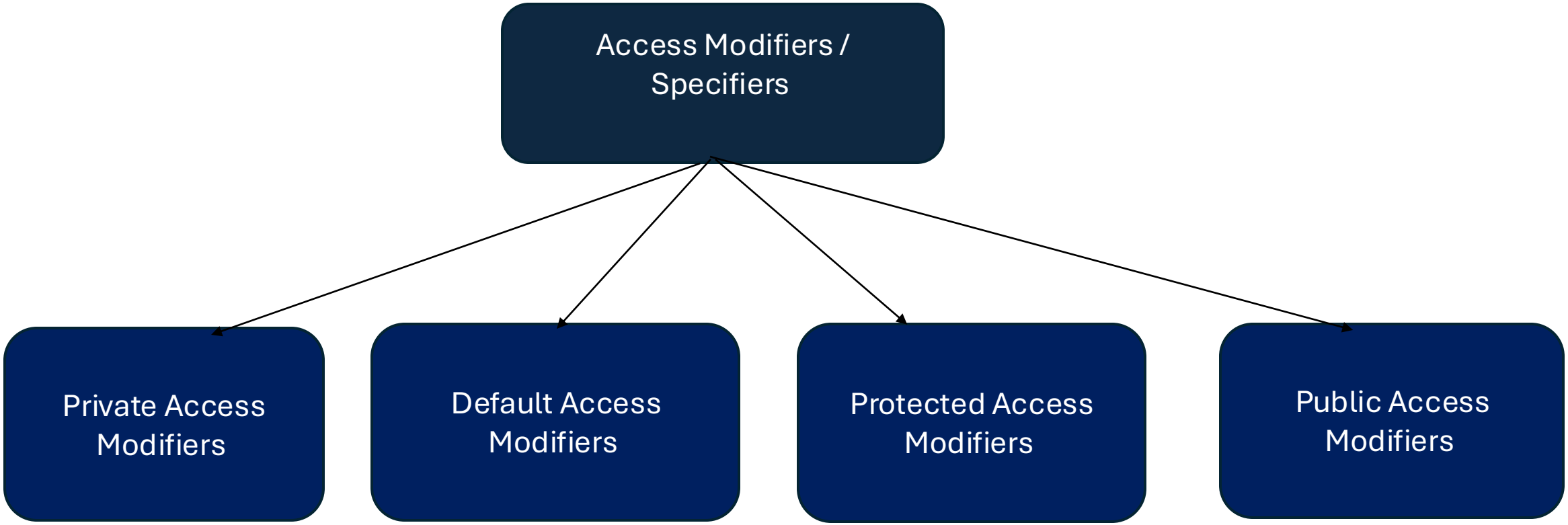
Topic:-02
"Access specifiers and Encapsulation"

Access Specifiers :-

- Access Specifiers are Also Known as Access Modifier.
- It is Used to Control Visibility and Accessibility of Classes and Methods.
- They help enforce Encapsulation by restricting access to different parts of program.

Types of Access Modifiers :-

In Java there are mainly four types of Access Specifiers .


Access Modifiers / Specifiers
Private Access Modifiers
Default Access Modifiers
Protected Access Modifiers
Public Access Modifiers

- **Private Access Modifiers/specifiers :-**
- The private access modifier is specified using the keyword private.
- The methods or data members declared as private are accessible only within the class in which they declared .

Example:-

```
Public class Account
{
    Private double balance;
    Public void deposit(double amount)
    {
      If(amount > 0)
      {
          Balance += amount;
      }
    }
  }
```

- **Default Access Modifiers / Specifiers :-**
- If no Access Modifiers is Specified , the member has default (package-driver) access and can only be accessed within the same package .
- This means only classes within the same package can access it .

Example:-

Package A

```
Class Car
{
  Int brand = "Suzuki";
   Void run()
    {
        System.out.println("Average of Suzuki Car is Good ");
     }
  }
```

- **Protected Access modifiers/ Specifiers:-**
- The protected access modifier is specified using the keyword protected.
- Protected access modifier in Java specifies that a member (method, variable, or constructor) is accessible within its own package and by subclasses in other packages.

  Example:-

  Parent Class (Package A)

  Package package_a

  Public class Parent {

  Protected Void displaymessage()

  {

  System.out.prinln("displaymessage from ParentClass");

  }

  }

SubClass (Package B)

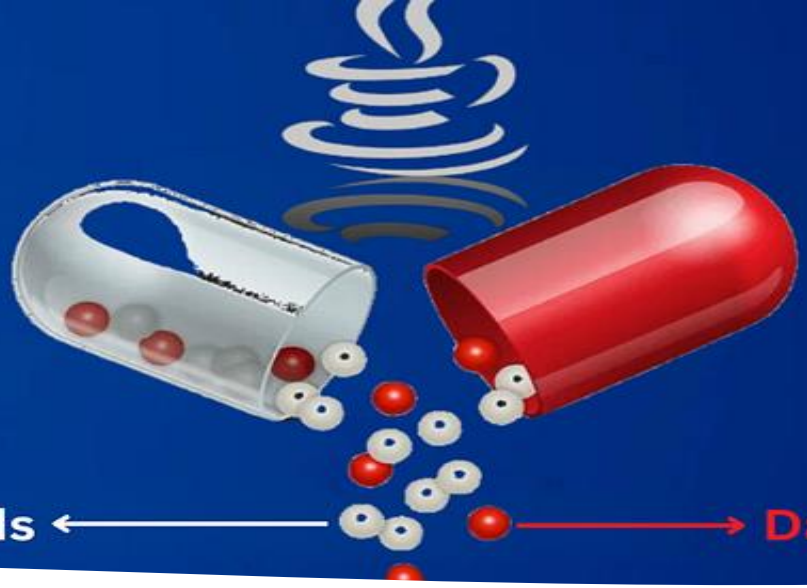
```
Package package_b
 Import package_a.parent;
 Public class Child extends parent
{
  Public static void main(String[] args)
  {
    Child childobj = new Child();
    Childobj.displaymessage();
   }
}
```

- **Public Access Modifiers/Specifiers :-**
- The public access modifier is specified using the keyword public.
- Public members are accessible from everywhere in the program. There is no restriction on the scope of public data members.

Example:-

```
Public class areacalc
{
  Public double CalculateSquarearea(double size)
  {
    Return side * side;
   }
}
```

Encapsulation In Java



- Encapsulation:
- Encapsulation is the one of the four fundamental principle of Object-Oriented programming (OOP).
- It process of binding / bundling data and methods into a single unit typically a class .
- Achieved Using Access Modifiers like Private, Protected and public.
- Improves Data Security by allowing Validation through Getter and Setter methods.

How Encapsulation Achieve in Java:

- Private Data Member.
- Public getters and setter's methods.

:- "Getter and Setter methods ":-

Think of **Getters** and **Setters** as the **security guards** of your Java class. When you make class variables `private` (so outside code can't touch them directly), getters and setters are the official `public` doors you create so other parts of your program can safely interact with that data.

- **Getter (`get`):** A method that **reads** and returns the value of a private variable.
- **Setter (`set`):** A method that **updates** or modifies the value of a private variable.

Example:-

```
public class Students
{
   public static void main(String[] args)
   {
     Student s1 = new Student();
     s1.setName("Dhruvil");
     s1.setAge(22);
```

```
System.out.println("Student Name: " + s1.getName());
System.out.println("Student Age: " + s1.getAge());
}
}
```

Topic :-03 "Inheritance and Types of Inheritance"

- **Inheritance** is one of the four main principles of Object-Oriented Programming (OOP).
- It allows one class (called the **child class** or **subclass**) to **inherit the properties (fields) and behaviors (methods)** of another class (called the **parent class** or **superclass**).
- " Extends " Keyword is used for inheritance in java .

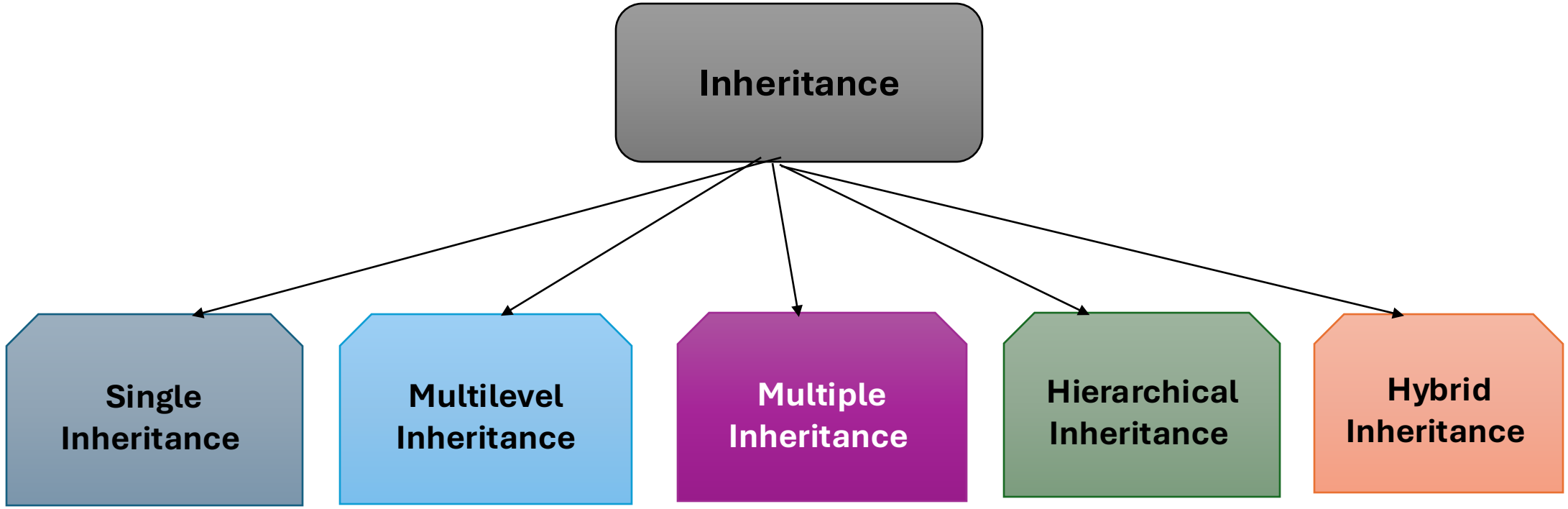


Example:-

```
class Vehicle // Parent class
{
   double speed;
   void startEngine()
   {
       System.out.println("Engine started.");
    }
 }
class Car extends Vehicle  // child class
{
  int numberOfDoors; // Unique property of Car
   void honk()
  {
   System.out.println("Beep beep!");
  }
}
```

```
public class Main
{
  public static void main(String[] args)
  {
    Car myCar = new Car();
    myCar.startEngine();
    myCar.honk();
  }
}
```

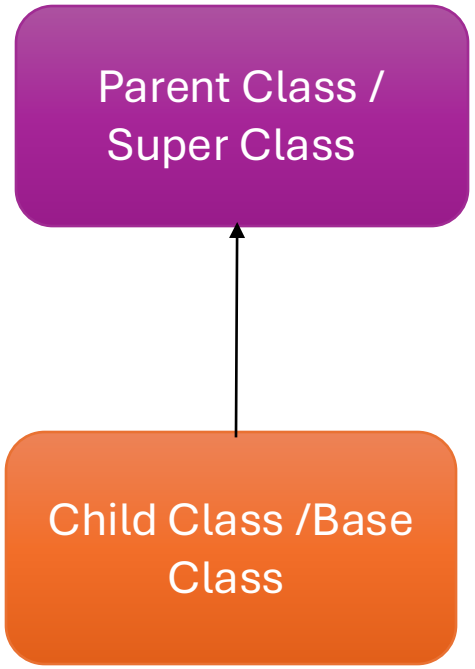
"Types of Inheritance "



• Single Inheritance :

- In single inheritance, a sub-class is derived from only one super class. It inherits the properties and behavior of a single-parent class.

Diagram :



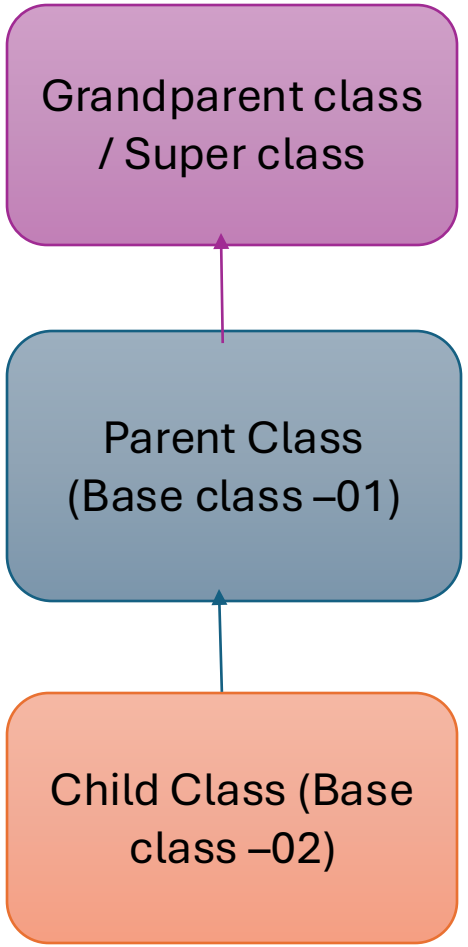
Example :

```
Class Vehicle { // parent class
Vehicle()
{
System.out.println(" This is Vehicle Class ");
}
}
Class Car extends Vehicle { // child class
Class()
{
System.out.println("This Vehicle is car ");
}
}
Public class test {
Public static void main(String[] args)
{
Car obj = new car();
Obj.class();
Obj.vehicle();
}
}
```

• Multilevel Inheritance :

- In Multilevel Inheritance The base class will derived properties and data from parent class and again parent class inherit all the data and methods from another super class .

Diagram :



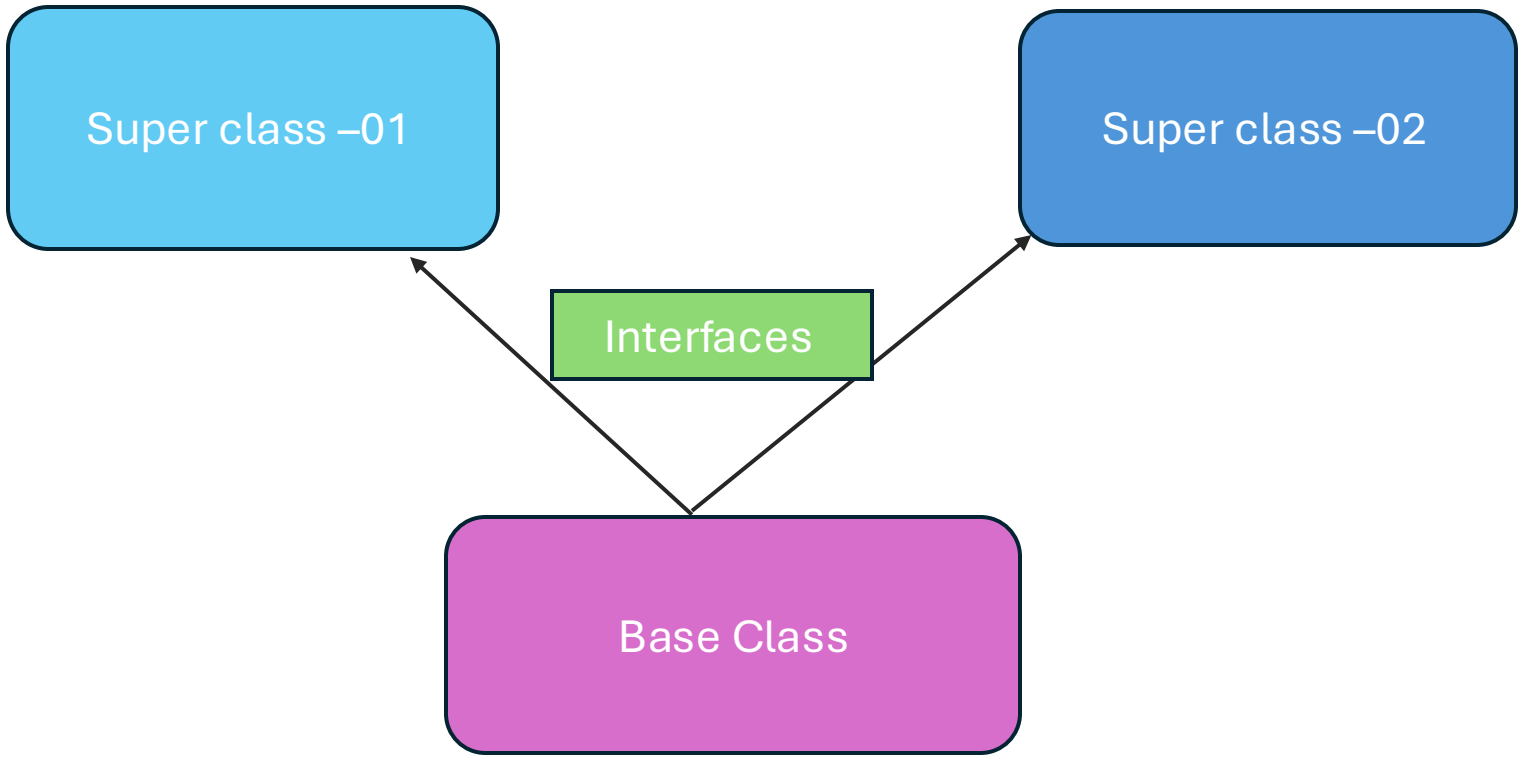
Example :

```
Class Vehicle {
Vehicle() {
System.out.println("This is grandparent Super class " );
}}
Class Fourwheeler extends Vehicle {
Fourwheeler(){
System.out.println("This is parent class –02 (Base Class –01)");
}}
Class  Car extends Fourwheeler {
Car() {
System.out.println("This is base class –02");
}}
Public class Final {
Public static void main(String[] args) {
Car obj = new Car();
Obj.car();
Obj.Fourwheeler();
Obj.vehicle();
}}
```

• Multiple Inheritance:

- In Multiple Inheritance There are two super class and one base class .
- When single base class inherit all the properties (methods and data) from two parent class is known as multiple inheritance.

Diagram :



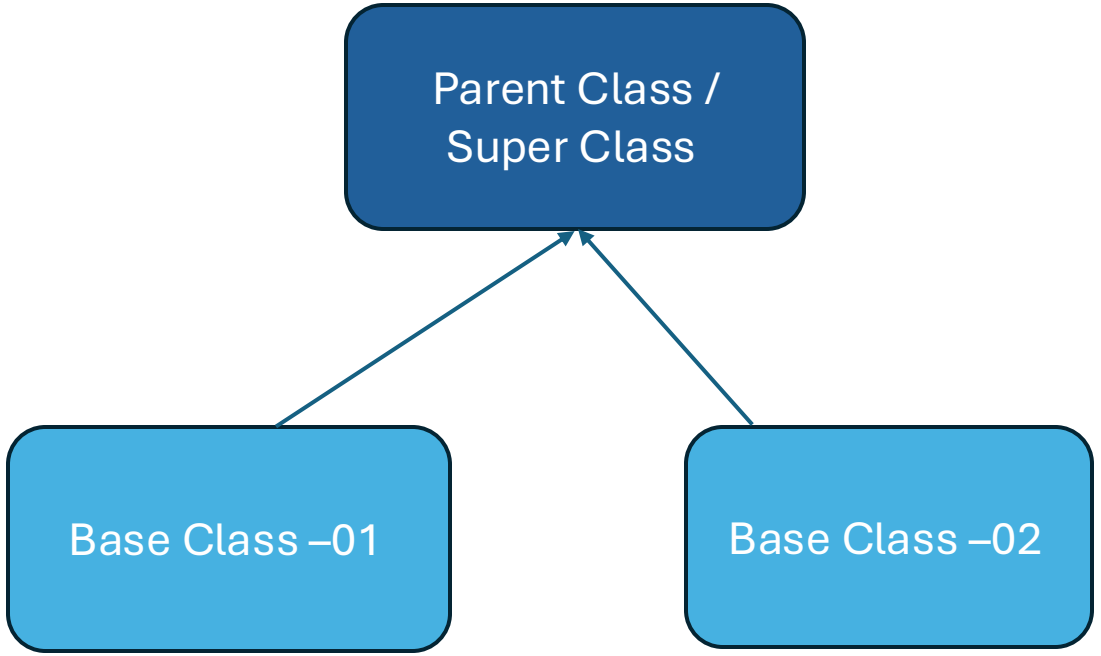
Example:

```
Interface vehicle {
Default void VehicleInfo() {
System.out.println("Vehicle Interface have  been implemented ");
}}
Interface Vehicletype {
Default void Vehicletype() {
System.out.println("Vehicletype Interface have been implemented");
}}
Class Autonomusvehicle implements vehicleinfo , vehicletype {
Autonomusvehicle() {
System.out.println(" base class inherit properties of both super class through interface");
}}
Public class test {
Public static void main(String[] args) {
Autonomusvehicle Automatic = new Autonomusvehicle();
Automatic.Vehicletype();
Automatic.VehicleInfo();
}}
```

• Hierarchical Inheritance :

- In Hierarchical Inheritance there are two base class and one super class.
- When two base class inherit all the properties (methods and Data) of one parent class then it is hierarchical inheritance.

Diagram :



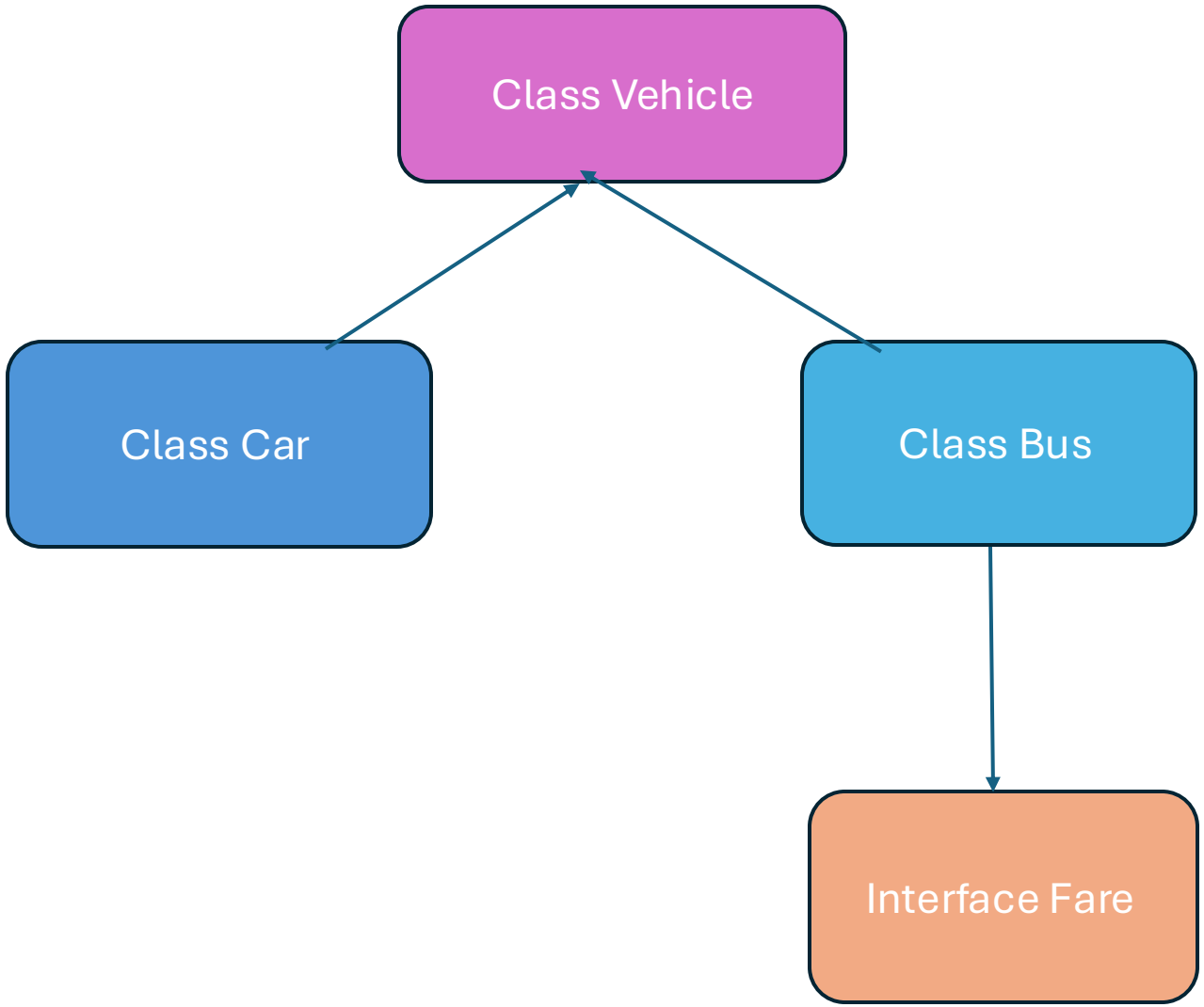
Example:

```
Class Vehicle {
Vehicle() {
System.out.println(" This is parent class ");
}}
Class maruti extends vehicle {
Maruti() {
System.out.println("This is base class-01 inherit super class properties");
}}
Class RR extends vehicle {
RR() {
System.out.println("This is base class-02 inherit same super class prop. ");
}}
Public class Final  {
Public static void main(String[] args) {
maruti obj1 = new maruti();
RR obj 2 = new RR();
}}
```

• Hybrid Inheritance :

- Hybrid Inheritance is an type of inheritance which is mix up of two or more of the above types of inheritance.

Example :



- Example :

```
Class Vehicle {
Void vehicleType() {
System.out.println(" This is Super class " );
} }
Interface fare {
Deault void fareInfo () {
System.out.println(" This is FareInfo ");
Class car extends vehicle {
Void car type() {
System.out.printlnI("This is category of car type");
} }
Class bus extends vehicle implements Fare {
Void bus type() {
System.out.prinln("This is Vehicle Category Bus " );
} }
```

```
Public class showing
{
Public static void main(String[] args) {
  Car C1 = new Car();
  C1.vehicletype();
  C1.car type();
  Bus B1 = new Bus();
  B1.Vehicletype();
  B1.bus type();
  B1.fareInfo();
}}
```

Topic:-04 "Polymorphism and Types of Polymorphism"

- Polymorphism is one of the fundamental principle in OOP .
- It Allows Single entity to take multiple forms and this is known as polymorphism.
- It enables the same methods or interface to behave differently depending on the object Involved .

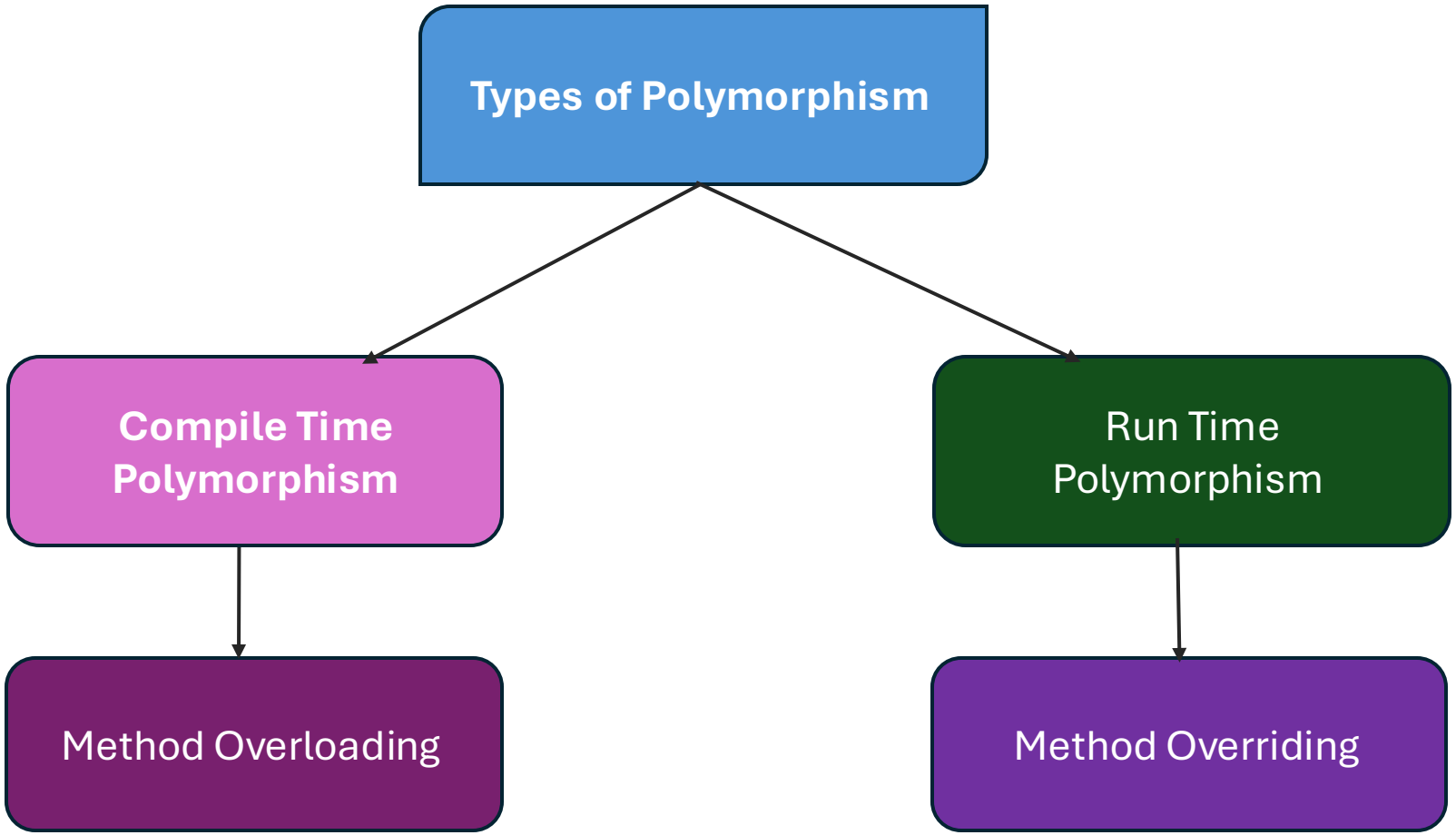
Diagram :



Example :-

```
Class person {
    Void role()
    {
     System.out.println("I am Person ");
    }
}
Class Employee extends person {
   @Override Void role()
    {
     System.out.println("I am Employee");
    }
}
Class main {
Public static void main(String[] args) {
Person p = new Employee ();
p.role();
}}
```

"Types of polymorphism"



- **Compile – Time Polymorphism :**
- Compile – Time Polymorphism also known as static polymorphism .
- The Compiler Determines Which method or operation to execute during compilation , before program runs.
- It is Primarily Achieved Through Method Overloading.

Method Overloading : Multiple Methods in the same class share the exact same name but have different parameter signature (different number , types of parameter or order of parameter.

Example :

```
Class Example {
    Static Void addition(int a , int b)
    {
        S.O.P("Addition of two number is : " +(a+b));
    }
    Static Void addition(int a , int b, int c)
    {
      S.O.P("Addition of three number is : " +(a+b+c));
     }
}
```

```
    Public static void main(String[] args)
     {
       int a=10;
       int b=20;
       int c=30;
      Addition(a , b);
     Addition(a, b, c);
    }
}
```

- **Run – Time Polymorphism :**
- Run – Time Polymorphism is also known as dynamic Polymorphism.
- The Decision of which method to Execute is postponed until the Program is actively running.
- It is Achieved through method overriding.

Method Overriding : A Child Class(Sub Class) Provides its own specific implementation of a method that is already defined in its parent class (Super class) . The method name , return type , parameters must match .

Example :

```
Class student {
Void Responsibilities() {
S.O.P(" To perform Day to Day Operations ");
}}
Class Attendance extends students {
@override
Void Responsibilities() {
S.O.P("Checking Attendance");
}
}
```

```
Class Assignments Extends Students {
   @override
    Void Responsibilities() {
     S.O.P("Complete Assignment");
      }}
     Public class main {
      Public static void main(String[] args) {
        Student s1 = new Attendance();
        Student s2 = new Assignments();
        S1.Responsibilities();
        S2.Responsibilities();
        }
      }
```

Topic :-05
"Abstraction , Abstract classes and Interfaces "

Abstraction :

- Abstraction is one of the fundamental concept .
- The Process of **hiding complex internal implementation details and showing only the essential features** of an object to the outside world.

Diagram :

Java

raction



Ways to Achieve Abstraction :

- ❖ Interface.
- ❖ Abstract Classes.

Example:

```
abstract class TV {
   abstract void turnon();
   abstract void turnoff();
 }
  Class TVRemote extends TV {
     @override
      Void turnon() {
        System.out.println("TV is Turned on ");
      }
     @override
      Void turnoff {
         System.out.println("TV is Turned off ");
      }
}
```

```
public class main {
    public static void main(String[] args)
    {
        TV Remote = new TVRemote();
        Remote.turnon();
        Remote.turnoff();
    }
}
```

" Abstract Classes "

- An Abstract Class in java is a restricted class that cannot be used to create objects directly .
- To use an abstract class , it must be inherited (extended) by another class.
- Declared with abstract keywords.
- Cannot be instantiated : Attempting new AbstractClass() will throws an compilation error.
- Can Contain abstract methods (methods without body).
- Can Contain Concrete methods (Normal methods with actual code blocks that child class automatically inherit .

Example :

```
abstract class animal {
    abstract void makesound();
     Void sleep() {
     System.out.println("Zzz....");
    }
}
```

```
Class Dog extends animal {
   @override
    Void makesound() {
    System.out.println("woff!");
   }
}
public class main {
     public static void main(String[] args) {
        Animal myDog = new Dog();
        myDog.makesound();
        myDog.sleep();
      }
}
```

Interface in Java

Interfaces "

- ✓ An Interface in java is blueprint that defines set of methods a class must implemented without providing full implementation details.
- ✓ It is used to achieve 100% Abstraction and multiple inheritance in java .
- ✓ A Class must implement all abstract methods of an interface.



ATM Machine



Example:

```
Interface Vehicle {
   Void start();
 }
 Class car implements vehicle {
     @override
      Public void start()
      {
         System.out.println("car start with key ignition");
      }
  }
 Class Bike implements vehicle {
   @override
    Public void start()
    {
       System.out.println("Bike start with a kick start");
```

```
}
}
 Public class main {
   Public static void main (String[] args) {
       Vehicle mycar = new car();
       Vehicle mybike = new bike();
       Mycar.start();
       Mybike.start();
   }
}
```

Topic :-06
" This , Super , Final Keywords "

This Keyword :

- "This" keyword refers to the current object inside a method or constructor

Example : //showing how the "This" keyword refers current object

```
Public class A {
Void show() {
System.out.println(this);
}
Public static void main(String[] args) {
A r = new A();
System.out.println(r);
r.show();
} }
```

Output :

A@1dbd16a6

A@1dbd16a6

- Whenever the name of instance variable and local variable both are same then our Runtime environment JVM get confused that which one is instance variable and which one is local variable to avoid this situation we use "This" keyword.

Example: //showing how This keyword is used

```
Class A {
   Int a ; // instance variable
     A(int a) //local variable
    {
      a=a;
    }
    Void show()
    {
     System.out.println(a);
   }
```

```
Public static void main(String[] args)
 {
   A r= new a(100);
   r.show();
 }
}
```

Output :- 0

After Changing specific statement

a=a to this.a=a;

Output:-100;

"Super Keyword "

- The super Keyword is a reference variable used by child class (Base class) to Quickly point to its immediate parent class (Super class).
- Ensures proper inheritance behavior and code reusability.
- Access Parent class methods and call parent class constructor.

Example :

```
Class Animal {
    String type = "Generic Animal";
    Animal(String type)
    {
      this.type= type;
 }}
 Class Dog extends Animal {
   String type = "Dog";
    Dog()
    {
```

```
   Super("canine") //call Animal Constructor
   }
    Void printTypes()
    {
      System.out.println(type); //print dog
      System.out.println(super.type); //print canine
    }
}
Public class main
{
    Public static void main(String[] args)
    {
        Dog d = new Dog();
         d.printTypes();
    }
}
}
```

Final Keyword :

- Final Keyword is used to restrict changes and make code more secure and predictable.
- It can be applied to variables , methods , and classes to prevent modifications , overriding , inheritance .
- Final Variable cannot be changed once assigned.
- Final method cannot be overridden.
- Final class cannot be inherited.

Example:

Final Variable

```
Public class main {
    Public static void main(String[] args)
    {
        Final double pi=3.14159;
        Pi=3.14;
        System.out.prinln("Value of pi will be :" +pi);
    }
}
```

Final method

```
Public class securityconfig {
Public final void validateToken() {
System.out.println("Validating token through securityconfig");
}
Public void logActivity() {
System.out.println("Default Logging Activity");
}}
Class customsecurity extend securityconfig {
@override
public void validateToken() {
System.out.println("Attempting custom token validation..");
}
@override
Public void logActivity() {
System.out.println("custom logging activity in customsecurity");
}}
```

```
Public class main {
  Public static void main(String[]args) {
    Customsecurity customsec = new customsecurity();
    Customsec.validateToken();
    Customsec.logActivity();
   }
}
```